

Лабораторная работа №1

Нагрузочное тестирование веб-приложения с использованием k6

Цель работы:

- понимать назначение и место нагрузочного тестирования в жизненном цикле разработки;
- использовать инструмент k6 для генерации нагрузки;
- измерять метрики производительности: RPS, latency, ошибки;
- интерпретировать перцентили (p50, p90, p95, p99) и отличать их от среднего;
- проводить сравнительный анализ производительности при различной интенсивности нагрузки;
- определять точку насыщения (saturation point) и момент начала деградации системы;
- формулировать технические требования к производительности.

Нагрузочное тестирование – это процесс проверки поведения системы под ожидаемой или пиковой пользовательской нагрузкой. В отличие от функционального тестирования, которое отвечает на вопрос «работает ли?», нагрузочное тестирование отвечает на вопрос «как быстро и сколько выдерживает?».

Основные метрики производительности

RPS – количество запросов, которое система успешно обрабатывает за одну секунду.

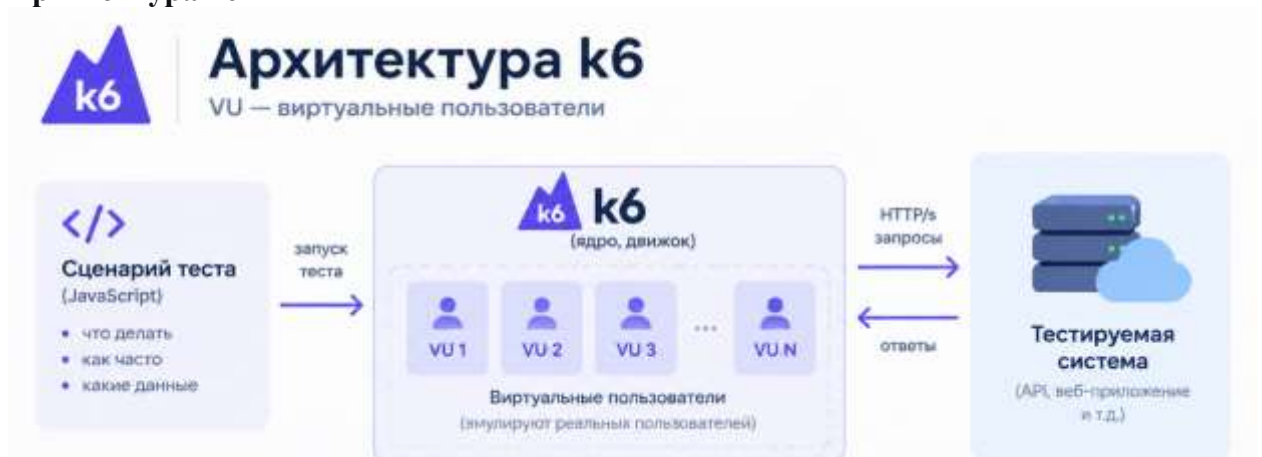
Latency – время от отправки запроса до получения ответа. Это время "ожидания" для конкретного пользователя.

Percentiles (Перцентили) – это статистические метрики, показывающие распределение задержек. – это статистические метрики, показывающие распределение задержек.

Error Rate (Частота ошибок) – Процент запросов, завершившихся ошибкой (HTTP 5xx, timeout, connection refused).

k6 – это инструмент для нагрузочного тестирования с открытым исходным кодом.

Архитектура k6



Virtual User (VU) – виртуальный пользователь, выполняющий сценарий (последовательность HTTP-запросов) в бесконечном цикле или заданное количество итераций.

Как k6 создаёт нагрузку:

1. Сценарий на JavaScript
2. k6 запускает N виртуальных пользователей
3. Каждый VU выполняет сценарий с заданной интенсивностью
4. k6 собирает метрики всех запросов (latency, status codes, RPS)
5. По окончании теста выводится статистика

Основные метрики k6

<i>Метрика</i>	<i>Описание</i>
http_reqs	Общее количество HTTP-запросов
http_req_duration	Полное время запроса (включая DNS, TCP, TLS, отправку, ожидание, получение)
http_req_waiting	Время ожидания ответа от сервера
http_req_failed	Количество неуспешных запросов
iterations	Количество выполненных итераций сценария
vus	Текущее количество виртуальных пользователей
checks	Результаты проверок

Подготовка окружения

В качестве тестируемого приложения можно взять веб-сервер, развёрнутый в Kubernetes.

Проверка доступности:

Проверяем, что Pod'ы работают

```
kubectl get pods
```

Проверяем, что Service создан

```
kubectl get services
```

Получаем IP сервиса (или используем port-forward)

```
kubectl get service web-app-service
```

Тестируем доступность

```
curl http://<service-ip>:<port>
```

или

```
kubectl port-forward service/web-app-service 8080:80
```

затем в браузере: http://localhost:8080

Ожидаемый результат: Страница приветствия веб-сервера (Nginx, Apache или ваше приложение).

Установка k6

Windows

```
winget install k6
```

Linux (Ubuntu/Debian)

```
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys  
C5AD17C747E3415A3642D57D77C6C491D6AC1D69
```

```
echo "deb https://dl.k6.io/deb stable main" | sudo tee
```

```
/etc/apt/sources.list.d/k6.list
```

```
sudo apt-get update
```

```
sudo apt-get install k6
```

Проверка установки

```
C:\Temp>k6 version
```

```
k6.exe v2.2.0 (commit/00a9a1b7f5, go1.26.5, windows/amd64)
```

Первый нагрузочный тест

Шаг 1. Создание простейшего сценария

Создадим файл `test-basic.js`:

```
test-basic.js - Блокнот
Файл Правка Формат Вид Справка
import http from 'k6/http';
import { check } from 'k6';
import { sleep } from 'k6';

// Базовый сценарий без настройки опций
export default function () {
  // Выполняем GET-запрос к нашему приложению
  const res = http.get('http://localhost:8080');

  // Проверяем, что ответ успешный
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 500 ms': (r) => r.timings.duration < 500,
  });

  // Пауза 1 секунда между запросами (эмуляция реального пользователя)
  sleep(1);
}
```

`http.get()` – отправляет GET-запрос

`check()` – выполняет проверки результаты попадут в отчёт

`sleep(1)` – останавливает выполнение VU на 1 секунду

По умолчанию k6 запускает 1 VU на 1 итерацию

Шаг 2. Запуск теста

`k6 run test-basic.js`

Шаг 3. Разбор вывода k6

```
TOTAL RESULTS
checks_total.....: 2      1.936755/s
checks_succeeded...: 100.00% 2 out of 2
checks_failed.....: 0.00%  0 out of 2

@ status is 200
@ response time < 500 ms

HTTP
http_req_duration.....: avg=17.02ms min=17.02ms med=17.02ms max=17.02ms p(90)=17.02ms p(95)=17.02ms
{ expected_response:true }...: avg=17.02ms min=17.02ms med=17.02ms max=17.02ms p(90)=17.02ms p(95)=17.02ms
http_req_failed.....: 0.00% 0 out of 1
http_reqs.....: 1      0.968378/s

EXECUTION
iteration_duration.....: avg=1.03s   min=1.03s   med=1.03s   max=1.03s   p(90)=1.03s p(95)=1.03s
iterations.....: 1      0.968378/s
vus.....: 1      min=1      max=1
vus_max.....: 1      min=1      max=1

NETWORK
data_received.....: 853 B 826 B/s
data_sent.....: 70 B 68 B/s
```

Основные метрики для анализа:

Метрика	Значение	Интерпретация
http_reqs	1	Количество выполненных запросов
http_req_duration{avg}	17.02 ms	Среднее время запроса
http_req_duration{p90}	17.02 ms	90% запросов быстрее 17.02 мс
http_req_duration{p95}	17.02 ms	95% запросов быстрее 17.02 мс
http_req_failed	0.00%	Нет ошибок

Почему не был использован параметр iterations?

`iterations` – это количество полных циклов сценария. Если в сценарии один запрос, `iterations` $\approx$ `http_reqs`. Но если в сценарии 5 запросов, `iterations` показывает количество выполнений сценария, а `http_reqs` – общее число запросов. Для расчёта RPS мы используем `http_reqs`.

Тест с фиксированной нагрузкой

В этом тесте зададим постоянное количество виртуальных пользователей на определённое время.

Шаг 1. Сценарий с фиксированными VU

Создадим файл `test-constant.js`:

```
test-constant.js - Блокнот
Файл Правка Формат Вид Справка
import http from 'k6/http';
import { check, sleep } from 'k6';

// Настройки нагрузки
export const options = {
  vus: 10,           // 10 виртуальных пользователей
  duration: '30s',   // тест длится 30 секунд
};

export default function () {
  const res = http.get('http://localhost:8080/');

  check(res, {
    'status is 200': (r) => r.status === 200,
  });

  // Реалистичная пауза между запросами
  sleep(1);
}
```

Шаг 2. Запуск теста

`k6 run test-constant.js`

Шаг 3. Анализ результатов

Заметим изменения по сравнению с первым тестом:

```
■ TOTAL RESULTS

checks_total.....: 300      9.893684/s
checks_succeeded...: 100.00% 300 out of 300
checks_failed.....: 0.00%   0 out of 300

■ status is 200

HTTP
http_req_duration.....: avg=8.93ms min=1.64ms med=4.2ms max=116.65ms p(90)=11.05ms p(95)=13.71ms
{ expected_response:true }...: avg=8.93ms min=1.64ms med=4.2ms max=116.65ms p(90)=11.05ms p(95)=13.71ms
http_req_failed.....: 0.00%   0 out of 300
http_reqs.....: 300      9.893684/s

EXECUTION
iteration_duration.....: avg=1.01s min=1s      med=1s      max=1.14s    p(90)=1.01s  p(95)=1.01s
iterations.....: 300      9.893684/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 256 kB 8.4 kB/s
data_sent.....: 21 kB 693 B/s

Running (0m30.3s), 00/10 VUs, 300 complete and 0 interrupted iterations
default  [*****] 10 VUs 30s
```

Что изменилось?**1. RPS вырос – с ~1 RPS до ~10 RPS**

При 1 VU было 0.97 запросов/сек, теперь – 9.89 запросов/сек. Рост практически линейный ($\times 10$), потому что каждый из 10 виртуальных пользователей отправляет ~1 запрос/сек (в скрипте есть `sleep(1)`). Это говорит о том, что сервер пока не является узким местом и спокойно обрабатывает возросшую нагрузку.

2. Latency выросла, но незначительно

<i>Метрика</i>	<i>1 VU</i>	<i>10 VU</i>	<i>Изменение</i>
avg	17.02 ms	8.93 ms	уменьшилась
med	17.02 ms	4.2 ms	уменьшилась
max	17.02 ms	116.65 ms	выросла в ~7 раз
p90	17.02 ms	11.05 ms	уменьшилась
p95	17.02 ms	13.71 ms	уменьшилась

Средняя и медианная latency снизились, потому что при 1 VU измерение было единственным и, вероятно, включало «холодный» первый запрос (установка соединения, DNS, TLS). При 300 запросах медиана показывает «типичный» ответ – 4.2 мс.

3. Появилось распределение задержек – теперь видны p90 и p95

При первом запросе все перцентили равны единственному значению (17.02 мс), что бесполезно для анализа. Теперь:

- **p90 = 11.05 мс** – 90% запросов быстрее 11 мс
- **p95 = 13.71 мс** – 95% запросов быстрее 13.7 мс

Разрыв между med (4.2 мс) и p95 (13.7 мс) – в 3 раза. Это признак **«тяжёлого хвоста»** распределения: большинство запросов очень быстрые, но есть небольшая доля медленных.

4. Максимальная latency (max) – 116.65 мс → есть медленные запросы

При 1 VU максимум был 17 мс, теперь – 116.65 мс (в ~7 раз больше). Это классический признак конкуренции за ресурсы:

- запросы встают в очередь на сервере,
- периодически срабатывает GC (сборка мусора),
- возможны сетевые всплески или переключения контекста.

Постепенное увеличение нагрузки (Stages)

Постепенно будем наращивать нагрузку и наблюдать, как меняются метрики, то есть имитируем реальный сценарий, когда пользователи приходят постепенно. Плато – держим нагрузку постоянной, чтобы снять стабильные метрики. Ramp-down – корректное завершение, чтобы не «оборвать» запросы. Общая длительность = 9 стадий × 30 сек = 4 мин 30 сек.

Шаг 1. Сценарий с этапами нагрузки

Создать файл `test-stages.js`:

```
test-stages.js - Блокнот
Файл Правка Формат Вид Справка

export const options = {
  stages: [
    { duration: '30s', target: 10 }, // 30 секунд поднимаемся до 10 VU
    { duration: '30s', target: 10 }, // 30 секунд держим 10 VU
    { duration: '30s', target: 50 }, // 30 секунд поднимаемся до 50 VU
    { duration: '30s', target: 50 }, // 30 секунд держим 50 VU
    { duration: '30s', target: 100 }, // 30 секунд поднимаемся до 100 VU
    { duration: '30s', target: 100 }, // 30 секунд держим 100 VU
    { duration: '30s', target: 200 }, // 30 секунд поднимаемся до 200 VU
    { duration: '30s', target: 200 }, // 30 секунд держим 200 VU
    { duration: '30s', target: 0 }, // 30 секунд снижаем до 0
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // Порог: 95% запросов < 500 мс
    http_req_failed: ['rate<0.01'], // Порог: ошибки < 1%
  },
};

export default function () {
  const res = http.get('http://localhost:8080');

  check(res, {
    'status is 200': (r) => r.status === 200,
  });

  // Пауза 1 секунда
  sleep(1);
}
```

Примечание!

`thresholds` – автоматические проверки результатов теста:

`thresholds: {`

`http_req_duration: ['p(95)<500'], // 95% запросов должны быть < 500 мс`

`http_req_failed: ['rate<0.01'], // Ошибок должно быть < 1%`

`}`

Тест не просто смотрит, что получилось, а **проверяет** соответствует ли система заданным требованиям. Если пороги нарушены, тест завершится с ошибкой (не PASS, а FAIL).

Шаг 2. Запуск теста

```
k6 run test-stages.js
```

```

TOTAL RESULTS

checks_total.....: 21540 79.561282/s
checks_succeeded...: 100.00% 21540 out of 21540
checks_failed.....: 0.00% 0 out of 21540

status is 200

HTTP
http_req_duration.....: avg=5.32ms min=517µs med=4.46ms max=127.63ms p(90)=8.22ms p(95)=11.34ms
{ expected_response:true }...: avg=5.32ms min=517µs med=4.46ms max=127.63ms p(90)=8.22ms p(95)=11.34ms
http_req_failed.....: 0.00% 0 out of 21540
http_reqs.....: 21540 79.561282/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.12s p(90)=1s p(95)=1.01s
iterations.....: 21540 79.561282/s
vus.....: 5 min=1 max=200
vus_max.....: 200 min=200 max=200

NETWORK
data_received.....: 18 MB 68 kB/s
data_sent.....: 1.5 MB 5.6 kB/s

Running (4m30.7s), 000/200 VUs, 21540 complete and 0 interrupted iterations
default [=====] 000/200 VUs 4m30s

```

Результат выполнения:

RPS вырос – с ~10 RPS до ~80 RPS

Тест	VU (макс)	RPS	Запросов всего
Предыдущий	10	9.89	300
Текущий	200	79.56	21 540

Рост RPS в 8 раз при росте VU в 20 раз. Это значит, что пропускная способность растёт нелинейно – где-то на высоких VU начинается насыщение (очереди, конкуренция за CPU/сеть/БД).

Latency не выросла, а даже упала – сервер держит нагрузку

Метрика	10 VU	200 VU	Изменение
avg	8.93 ms	5.32 ms	лучше
med	4.2 ms	4.46 ms	стабильно
p(90)	11.05 ms	8.22 ms	лучше
p(95)	13.71 ms	11.34 ms	лучше
max	116.65 ms	127.63 ms	незначительно

Сервер отлично масштабируется до 200 VU. Медиана ~4.5 мс, p95 ~11 мс – это очень хорошие показатели. «Тяжёлый хвост» остался на том же уровне (max ~127 мс).

Thresholds пройдены – оба

```
THRESHOLDS
http_req_duration
  [pass] 'p(95)<500' p(95)=11.34ms
http_req_failed
  [pass] 'rate<0.01' rate=0.00%
```

Пороги выполнены с огромным запасом. Это значит, что целевой SLO (95% < 500 мс, ошибки < 1%) соблюдается даже при 200 VU.

Ошибки не появились: 0 ошибок из 21 540 запросов

Все 21 540 запросов вернули 200 ОК. Это говорит о стабильности сервера под нагрузкой.

vus – min=1, max=200

```
vus.....: 5      min=1      max=200
vus_max.....: 200    min=200    max=200
```

vus в момент завершения = 5 (тест уже шёл на ramp-down).

min=1 – старт с 1 VU (ramp-up с нуля).

max=200 – пик нагрузки достигнут.

iteration_duration – стабильно ~1 сек

```
iteration_duration.....: avg=1s    min=1s    med=1s    max=1.12s  p(90)=1s   p(95)=1.01s
iterations.....: 21540  79.561282/s
```

Итерация = 1 HTTP-запрос + sleep(1). Разброс минимальный (max 1.12 с), то есть даже под нагрузкой 200 VU пауза sleep(1) не меняется. Это подтверждает, что запросы обслуживаются быстро.

Задания для самостоятельного выполнения**Задание 1. Базовый нагрузочный тест**

Провести первичный анализ производительности.

Задачи:

1. Создать сценарий с 10 виртуальными пользователями на 30 секунд.
2. Добавить проверку статуса ответа (200 OK).
3. Запустить тест и зафиксировать метрики.

Необходимо:

- Предоставить полный вывод k6
- Выделить значения: RPS, p50, p95, p99, ошибки (%)
- Написать краткий анализ
-

Задание 2. Исследование влияния нагрузки на метрики

Проанализировать, как изменение числа VU влияет на ключевые метрики.

Задачи:

1. Провести серию тестов с VU: 10, 25, 50, 100, 200.
2. Для каждого теста зафиксировать:
 - RPS (`http_reqs / duration`)
 - p50, p90, p95, p99 latency
 - Процент ошибок
3. Заполнить таблицу.

Необходимо:

- Таблица со всеми значениями
- График зависимости p95 от VU
- Краткий анализ

Шаблон таблицы:

VU	RPS	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Errors (%)
10						
25						
50						
100						
200						

Задание 3. Поиск точки деградации

Определить, при какой нагрузке система начинает деградировать.

Задачи:

1. Увеличивать нагрузку (VU) до тех пор, пока не выполнится одно из условий:
 - p95 latency вырос более чем в 2 раза по сравнению с предыдущим шагом
 - Появились ошибки ($> 0.1\%$)
 - RPS перестал расти (прирост $< 10\%$ при удвоении VU)
2. Зафиксировать точку деградации.
3. Определить причину деградации.

Необходимо:

- График Latency vs Load с отмеченной точкой деградации
- Таблица
- Написать краткий анализ

Контрольные вопросы

1. Что такое нагрузочное тестирование и чем оно отличается от функционального?
2. Какие метрики производительности вы знаете? Дайте определение RPS, latency, перцентилей.
3. Почему среднее не является достаточной метрикой? Приведите пример.
4. Что означают p50, p95, p99? Как интерпретировать значение p99 = 500 мс?
5. Как работает k6? Что такое Virtual User ?
6. Как определить точку деградации системы? Какие признаки указывают на деградацию?
7. Что такое thresholds в k6 и зачем они нужны?
8. Как сформулировать корректное техническое требование к производительности?
9. Почему при увеличении нагрузки RPS может перестать расти?
10. Какие есть способы оптимизации производительности, если система не справляется с нагрузкой?